皖西学院·考点对应模式 · 答案与解析

一、简答题（4×5=20分）

1.

答案:

浮点数由阶码（含阶符）和尾数（含数符）两部分组成，在总位长固定的前提下:

（1）**表示范围由阶码的位数决定**（2分）。阶码位数越多，能表示的指数范围越大，浮点数的表示范围就越大。

（2）**精度由尾数的位数决定**（2分）。尾数位数越多，能表示的有效数字越多，精度越高。

（3）**权衡关系**：在总位长固定时，增加阶码位数可扩大范围，但会减少尾数位数从而降低精度；反之亦然。两者互相制约，设计时需根据应用需求折中。（1分）

解析：本题考察浮点数表示中范围与精度的概念辨析及权衡。常见错误：将范围与精度混淆，认为都由尾数或都由阶码决定；遗漏权衡关系的讨论。得分要点：阶码→范围、尾数→精度、权衡制约三个点缺一不可。

2.

答案:

按功能划分，系统总线分为三类： - **地址总线**：传送 CPU 访存的地址信息，单向（CPU→存储器/IO），其位数决定最大物理地址空间。 - **数据总线**：传送数据信息，双向，其位数通常与机器字长有关，决定一次总线传输的数据量。 - **控制总线**：传送控制信号和时序信号，包括读/写信号、中断请求、总线请求等。

总线复用：指同一条总线在不同时间分时传送不同类型的信息（如地址/数据复用），通过分时控制实现。

- **优点**：减少总线引脚数目，降低硬件成本和芯片封装复杂度。
- **缺点**：降低了总线的数据吞吐率（同一时刻只能传一类信息），需要额外的控制逻辑（如 ALE 锁存信号）来区分时段。

解析：本题考察总线按功能分类（地址/数据/控制三大类）以及总线复用的定义与利弊。常见混淆：将总线复用与总线的"总线仲裁"混淆；遗漏复用需要分时控制的说明。

3.

答案:

"程序员可见"是指该寄存器可以由程序员通过指令直接或间接访问和操作（即可用指令指定其作为操作数或目标）。

- **可见的寄存器**：通用寄存器（如 R0~Rn）、程序计数器（PC）、程序状态字寄存器（PSW）、累加器（ACC）等。程序员可以通过指令直接引用或间接使用它们。
- **不可见的寄存器**：指令寄存器（IR）、存储器地址寄存器（MAR）、存储器数据寄存器（MDR）、暂存器等。这些寄存器是 CPU 内部执行指令时自动使用的，程序员无法用指令直接访问。

解析：本题考察 CPU 内部寄存器的分类与程序员可见性概念。关键区分点：可见寄存器是程序员能通过编程控制的；不可见寄存器属于 CPU 微观实现细节。常见错误：认为 IR 或 MAR 对程序员可见（实际不可见），或认为 PC 不可见（实际可见，转移类指令直接操作 PC）。

4.

答案：

- **机器字长**：CPU 一次能处理的二进制数据的位数，通常等于 ALU 的位数和通用寄存器的宽度，反映了 CPU 的数据处理能力。
- **存储字长**：主存的一个存储单元所能存放的二进制数据的位数，即每个存储地址对应的数据宽度。
- **指令字长**：一条机器指令的二进制位数。指令字长可以是固定的（等于机器字长），也可以是变长的（半字长、单字长、双字长等）。

三**者的关系**：三者**不一定相等**。在许多机器中指令字长可以小于、等于或大于机器字长（变长指令集）；存储字长通常与机器字长一致以方便存取，但并非必然，在按字节编址的系统中存储字长可以是 8 位而机器字长为 32 位。三者是不同层面（处理能力、存储宽度、指令编码）的概念，没有必然的相等关系。

解析：本题考察三种"字长"的定义及关系辨析。常见错误：认为三者必然相等，或混淆指令字长与机器字长。关键是理解三者分别属于不同的概念层面——机器字长衡量处理能力、存储字长衡量存储单元宽度、指令字长衡量指令编码长度。

二、计算题（4×10=40分）

1.

已知参数

参数	值
程序总指令数 N	10^8 = 100,000,000 条
指令占比	I_1=40%, I_2=30%, I_3=20%, I_4=10%（占比和=100%）

参数	值
机器 A	f = 500 MHz；CPI: I_1=1, I_2=3, I_3=4, I_4=6
机器 B	f = 750 MHz；CPI: I_1=2, I_2=3, I_3=3, I_4=5

第 1 问：加权平均 CPI

公式：**加权 CPI = Σ(CPI_i × **$占比_i$**)**

机器 A

$CPI_A = 1 \times 0.4 + 3 \times 0.3 + 4 \times 0.2 + 6 \times 0.1 = 0.4 + 0.9 + 0.8 + 0.6 = 2.70$

机器 B

$CPI_B = 2 \times 0.4 + 3 \times 0.3 + 3 \times 0.2 + 5 \times 0.1 = 0.8 + 0.9 + 0.6 + 0.5 = 2.80$

> **验证**：占比归一 0.4+0.3+0.2+0.1 = 1.0 ✓

第 2 问：MIPS

公式：**MIPS = 时钟频率(MHz) / 加权 CPI**

机器 A

$MIPS_A = 500/2.70 = 185.19$

机器 B

$MIPS_B = 750/2.80 = 267.86$

> 注意：时钟频率以 MHz 为单位直接代入即可（因 MIPS = 10^6 指令/秒，f 以 Hz 为单位除以 10^6 = 以 MHz 为单位）。

第 3 问：性能比较

公式：**执行时间 = (N × 加权 CPI) / 时钟频率(Hz)**

也可用：**执行时间 = N / (MIPS × 10^6)**

机器 A

- 公式 1：$T_A = 10^8 \times 2.70/500 \times 10^6 = 2.70 \times 10^8/5.0 \times 10^8 = \mathbf{0.54\ s}$
- 公式 2（验证）：$T_A = 10^8/185.19 \times 10^6 = \mathbf{0.54\ s}$ ✓（两种公式结果一致）

机器 B

- 公式 1：$T_B = 10^8 \times 2.80/750 \times 10^6 = 2.80 \times 10^8/7.5 \times 10^8 = \mathbf{0.3733\ s}$
- 公式 2（验证）：$T_B = 10^8/267.86 \times 10^6 = \mathbf{0.3733\ s}$ √

结论

机器 B 更快。

定量依据： - $T_B = 0.3733$ s < $T_A = 0.54$ s - 加速比 = $T_A / T_B = 0.54 / 0.3733 \approx 1.45$

机器 B 虽然加权 CPI 略高（2.80 > 2.70），但其时钟频率更高（750 MHz vs 500 MHz），综合效果是单位时间内执行更多指令（MIPS_B = 267.86 > MIPS_A = 185.19），执行时间更短。

结果汇总

指标	机器 A	机器 B
加权 CPI	2.70	2.80
MIPS	185.19	267.86
执行时间 (s)	0.5400	0.3733

机器 B 性能更优（快约 1.45 倍）。

场景规则验证

规则	结果
R1 占比归一（和=1.0）	√ True
R2 加权CPI = $\Sigma(CPI_i \times ratio_i)$	√ A=2.70, B=2.80
R3 MIPS = f(MHz)/CPI	√ A=185.19, B=267.86
R4 执行时间两公式一致	√ A: 0.54=0.54; B: 0.3733=0.3733
R5 执行时间不同，存在唯一更快者	√ B 更快
R6 参数合法（CPI∈[1,8], 频率∈[200,800]MHz, 频率不同, CPI不完全相同）	√ 全部 True

2.

补码加减运算与溢出判断 —— 参考解答

> 本解答数值均由 gen.py exec 验证产出，逐字对照数据流输出转写。

基本信息

参数	值
$x = -0.6875_{10} = -0.1011_2$	$\|x\| \times 2^4 = 11 = 1011_2$
$y = -0.5625_{10} = -0.1001_2$	$\|y\| \times 2^4 = 9 = 1001_2$
n=4 数值位，总位数 =6	双符号位格式：$S_f\, S_f\, .\, d_3\, d_2\, d_1\, d_0$

第 1 问（3 分）：求 [x]_补 和 [y]_补

求 [x]_补

x 为负数，原码双符号位格式为：

[x]_原 = 11.1011

对**数值部分**按位取反、末位加 1（符号位不变）：

步骤	操作	结果
数值部分	原码数值 1011	1011
按位取反	\sim 1011	0100
末位加 1	0100 + 1	0101

\boxed{[x]_补 = 11.0101}

求 [y]_补

y 为负数，原码双符号位格式为：

[y]_原 = 11.1001

对数值部分按位取反、末位加 1：

步骤	操作	结果
数值部分	原码数值 1001	1001

步骤	操作	结果
按位取反	\sim 1001	0110
末位加 1	0110 + 1	0111

\boxed{[y]_补 = 11.0111}

验证（SR1 补码可逆性）

- [x]_补 = 11.0101 的双符号位补码语义值 = $(110101_2 - 1000000_2)/2^4 = (53-64)/16 = -11/16 = -0.6875$ ✓
- [y]_补 = 11.0111 的双符号位补码语义值 = $(110111_2 - 1000000_2)/2^4 = (55-64)/16 = -9/16 = -0.5625$ ✓

第 2 问（4 分）：[x+y]_补 加法运算与溢出判断

逐位加法过程

\begin{aligned} [x]_补 &= 11.0101 \\ [y]_补 &= 11.0111 \\ \end{aligned}

从最低位到最高位逐位相加（含进位）：

位序	[x]_补	[y]_补	进位输入	本位和	向高位进位
d_0	1	1	0	**1**	1
d_1	0	1	1	**0**	1
d_2	1	0	1	**0**	1
d_3	0	1	1	**0**	1
S_{f0}	1	1	1	**1**	1
S_{f1}	1	1	1	**1**	1

最高位向外进位输出：**1**

运算结果

[x]_补 + [y]_补 = 10.1100

（6 位结果为 101100，最高位进位自然丢弃）

溢出判断

结果的双符号位 $S_{f1}S_{f0} = 10$。

- $S_{f1}S_{f0} = 10$：最高符号位与次高符号位不同，**发生溢出**。

· 两负数相加，结果 x + y = -0.6875 + (-0.5625) = -1.25 < -1，超出定点小数可表示范围 [-1, 1)，为**下溢（负溢出）**。

\boxed{[x+y]_补 = 10.1100,\quad 双符号位=10,\quad 下溢（负溢出），运算结果不可信}

第 3 问（3 分）：[x-y]_补 减法运算、溢出判断与规则分析

(1) 求 [-y]_补

[-y]_补 由 [y]_补 **连同符号位一起**按位取反再末位加 1（求补操作）：

[y]_补 = 11.0111

步骤	操作	结果
[y]_补（6位）	连同符号位	110111
按位取反	\sim 110111	001000
末位加 1	001000 + 1	001001

[-y]_补 = 00.1001

验证（SR4）：[y]_补 + [-y]_补 = 110111 + 001001 = 1000000，低 6 位 = 000000 = 00.0000 ✓

(2) 加法运算过程

\begin{aligned} [x]_补 &= 11.0101 \\ [-y]_补 &= 00.1001 \\ \end{aligned}

从最低位到最高位逐位相加：

位序	[x]_补	[-y]_补	进位输入	本位和	向高位进位
d_0	1	1	0	**0**	1
d_1	0	0	1	**1**	0
d_2	1	0	0	**1**	0
d_3	0	1	0	**1**	0
S_{f0}	1	0	0	**1**	0
S_{f1}	1	0	0	**1**	0

最高位向外进位输出：**0**

运算结果与溢出判断

[x-y]_补 = [x]_补 + [-y]_补 = 11.1110

双符号位 $S_{f1}S_{f0}$ = 11：两位符号位相同，**无溢出**。

语义值 = $(111110_2 - 1000000_2)/2^4$ = (62-64)/16 = -2/16 = -1/8 = -0.125，与 x - y = -0.6875 - (-0.5625) = -0.125 一致 √。

$\boxed{[x-y]_{补} = 11.1110,\quad 双符号位=11,\quad 无溢出，结果=-0.125}$

(3) 双符号位溢出判断规则与对比分析

双符号位 $S_{f1}S_{f0}$ 的四种组合：

$S_{f1}S_{f0}$	含义	是否溢出
00	结果为正数，无溢出	否
01	正溢出（上溢）：结果超过正上界 +1	**是**（上溢）
10	负溢出（下溢）：结果低于负下界 -1	**是**（下溢）
11	结果为负数，无溢出	否

判断规则：双符号位中 $S_{f1} \neq S_{f0}$（即 01 或 10）时溢出；$S_{f1} = S_{f0}$（即 00 或 11）时不溢出。

为何第 2 问溢出而第 3 问不溢出：

- **第 2 问**（x+y）：x 与 y **同号**（均为负数），|x+y| = |-0.6875 + (-0.5625)| = 1.25 > 1，绝对值超出定点小数表示范围 [-1,1)。两个负数相加，负方向"超出"，符号位本应一致为 11 却因进位变形为 10 → **下溢**。
- **第 3 问**（x-y = x+(-y)）：x 为负、-y 为正，是**异号相加**。|x-y| = |-0.6875+0.5625| = 0.125 < 1，绝对值在表示范围内。异号相加结果不会超出 [-1,1) 的范围 → **无溢出**。

本质：同号相加才可能溢出（绝对值增大），异号相加绝对值减小，不可能溢出。

3.

第 1 问（4 分）

（1）54.375 的二进制表示

对绝对值 **54.375** 分别转换整数部分和小数部分。

整数部分 54（除 2 取余）：

除法	商	余数
54 ÷ 2	27	0
27 ÷ 2	13	1
13 ÷ 2	6	1
6 ÷ 2	3	0

除法	商	余数
3 ÷ 2	1	1
1 ÷ 2	0	1

从下往上读余数：**$54 = 110110_2$**

小数部分 0.375（乘 2 取整）：

乘法	结果	取整
0.375 × 2	0.75	**0**
0.75 × 2	1.5	**1**
0.5 × 2	1.0	**1**

从上往下读整数：**$0.375 = 0.011_2$**

合并：

$54.375 = 110110.011_2$

（2）规格化形式 $1.xxx \times 2^E$

将二进制小数点左移，使整数部分仅剩 1 位且为 1：

$110110.011 = 1.10110011 \times 2^5$

- 整数部分 `110110` 共 6 位，左移 5 位使小数点位于第一个 1 之后。
- 去掉隐含前导 1 后的尾数有效位为 `10110011`。

（3）阶码真值 E

\boxed{E = 5}

（= 整数部分二进制位数 6 − 1）

第 2 问（6 分）

（1）符号位 S

输入为负数，故：

\boxed{S = 1}

（阶码和尾数取绝对值部分的规格化结果）

（2）阶码字段（E + 127 的 8 位二进制）

E + 127 = 5 + 127 = 132

$132 = 10000100_2$

\boxed{阶码字段 = 10000100}

（3）尾数字段（23 位二进制，不含隐含前导 1）

有效位去隐含 1 后为 `10110011`（8 位），低位补零至 23 位：

\boxed{尾数字段 = 10110011 0000000000000}

即 `10110011000000000000000`（恰好 23 位，无需舍入）。

（4）32 位完整机器数（S + 阶码 + 尾数）

字段	位	值
符号位 S	1 位	`1`
阶码字段	8 位	`10000100`
尾数字段	23 位	`10110011000000000000000`

拼接：

\boxed{32 位机器数 = 1 10000100 10110011000000000000000}

（5）最终的十六进制表示

将 32 位二进制每 4 位一组：

```
1100 0010 0101 1001 1000 0000 0000 0000
  C    2    5    9    8    0    0    0
```

\boxed{十六进制 = C2598000}

结果汇总

项目	结果
54.375 二进制	`110110.011`
规格化形式	1.10110011×2^5
阶码真值 E	5
符号位 S	1
阶码字段	`10000100`
尾数字段	`10110011000000000000000`

项目	结果
32 位机器数	`11000010010110011000000000000000`
十六进制	**C2598000**

4.

编号	虚拟地址	虚页号	页内偏移	装入位	物理地址 / 结论
①	0x000035A8	0x3	0x5A8	1	**0x3AB5A8**
②	0x000041F0	0x4	0x1F0	1	**0x1F1F0**
③	0x000067C4	0x6	0x7C4	1	**0xC7C4**
④	0x00002B3E	0x2	0xB3E	0	缺页，不可转换
⑤	0x00005A10	0x5	0xA10	0	缺页，不可转换

三、分析题（2×10=20分）

1.

Cache 全相联映射分析 —— 求解过程

问题 1（3 分）

Cache 行数、地址结构、Tag 位数

- Cache 行数：N = C / L = 2048 / 64 = **32 行**
- 全相联映射下，主存地址分为两部分：
- **Tag（标记）**：即主存块号，位数 = m = A − b = 16 − 6 = **10 位**
- **块内地址**：位数 = b = $\log_2(L) = \log_2(64)$ = **6 位**
- **Tag 占 10 位**（全相联映射中，Tag = 完整主存块号，无行索引位）

> 注意：全相联映射没有行索引（组索引）字段，整个块号就是 Tag。这与直接映射不同——直接映射会从块号中分出行号位作为索引。

问题 2（4 分）

逐一分析 6 个访存地址。块号 = addr >> 6（右移 6 位，去掉块内地址）。

序号	访存地址	块号(Tag)	判定	说明
①	0x0280	10	**命中**	块号 10 在初始 Cache 中（行 10 存放块 10）
②	0x0A00	40	**未命中**	块号 40 不在 Cache 中，装入**行 0**，替换**块 0**（FIFO 队首）
③	0x0500	20	**命中**	块号 20 在 Cache 中（行 20 存放块 20）
④	0x0C00	48	**未命中**	块号 48 不在 Cache 中，装入**行 1**，替换**块 1**（FIFO 队首）
⑤	0x0140	5	**命中**	块号 5 在 Cache 中（行 5 存放块 5）
⑥	0x1000	64	**未命中**	块号 64 不在 Cache 中，装入**行 2**，替换**块 2**（FIFO 队首）

FIFO 替换过程说明：

初始 FIFO 队列（按装入顺序）：[0, 1, 2, …, 31]，队首块 0 最先被替换。

- ② 未命中 → 替换队首块 0（行 0），装入块 40；FIFO 队列更新为 [1, 2, …, 31, 40]
- ④ 未命中 → 替换队首块 1（行 1），装入块 48；FIFO 队列更新为 [2, 3, …, 31, 40, 48]
- ⑥ 未命中 → 替换队首块 2（行 2），装入块 64；FIFO 队列更新为 [3, 4, …, 31, 40, 48, 64]

问题 3（3 分）

内容发生变化的行：

行号	原存放块号	现存放块号(Tag)
0	0	**40**
1	1	**48**
2	2	**64**

最终 Cache 各行块号：[40, 48, 64, 3, 4, 5, 6, …, 31]（行 3~31 未变化，仍存放块 3~31）。

命中率： 命中 3 次 / 总访问 6 次 = **50%**

2.

中断I/O方式可行性分析 —— 参考解答

第（1）问：处理一次中断所需的总时间 T_{total}（3 分）

中断处理分为三个阶段，所需时钟周期数之和为总周期数：

$C_{total} = C_{resp} + C_{serv} + C_{rest} = 500 + 4000 + 500 = 5000$ 个时钟周期

CPU 主频 $f = 2$ GHz $= 2 \times 10^9$ Hz，即每秒 2×10^9 个时钟周期。

$T_{total} = \frac{C_{total}}{f} = 5000/2 \times 10^9 = 2.5 \times 10^{-6}$ s

$\boxed{T_{total} = 2.5\ \mu s}$

> **关键点**：中断响应开销（保护/恢复现场各 500 周期）必须计入，不能只算中断服务的 4000 周期。三个阶段开销缺一不可。

第（2）问：设备 A、B 两次中断之间的最小时间间隔 T_A、T_B（3 分）

每次中断传输的数据量 D = 4 B。缓冲寄存器两次填满之间的时间间隔为：

$T_{interval} = D/R$

其中 R（B/s）= R（KB/s）× 1024。

设备 A（R_A = 512 KB/s）：

$R_A = 512 \times 1024 = 524288$ B/s

$T_A = 4/524288 = 7.62939453125 \times 10^{-6}$ s

$\boxed{T_A \approx 7.63\ \mu s}$

设备 B（R_B = 2048 KB/s）：

$R_B = 2048 \times 1024 = 2097152$ B/s

$T_B = 4/2097152 = 1.9073486328125 \times 10^{-6}$ s

$\boxed{T_B \approx 1.91\ \mu s}$

> **关键点**：单位换算链为 传输率（KB/s）→ 字节率（B/s，乘以 1024）→ 时间间隔（D/R，秒）→ 微秒（乘以 10^6）。注意 1 KB = 1024 B（二进制换算）。

第（3）问：中断 I/O 方式可行性判断（4 分）

不丢失数据的条件：$T_{total} \leqslant T_{interval}$（CPU 处理一次中断的总时间不超过两次中断之间的间隔时间）。

设备 A：可行

$T_{total} = 2.5\ \mu s \leqslant T_A = 7.63\ \mu s$

CPU 在缓冲寄存器下一次填满之前能完成中断处理，**不会丢失数据，采用中断 I/O 方式与设备 A 通信可行**。

安全裕度：$T_A / T_{total} = 7.63 / 2.5 \approx 3.05$，间隔时间是处理时间的约 3 倍，裕度充足。

设备 B：不可行

$T_{total} = 2.5\ \mu s > T_B = 1.91\ \mu s$

CPU 处理一次中断需要 2.5 μs，但设备 B 每隔 1.91 μs 就会产生一次中断请求（缓冲寄存器填满）。当 CPU 还在处理上一次中断时，下一次中断已经到来，缓冲寄存器中的数据会被覆盖，**造成数据丢失，采用中断 I/O 方式与设备 B 通信不可行**。

比值：$T_{total} / T_B = 2.5 / 1.91 \approx 1.31$，处理时间已超出间隔时间的 31%，无法及时处理。

改进方案：设备 B 数据传输率高、中断间隔短，应考虑改用 **DMA（直接存储器存取）方式**。DMA 方式下，数据在外设与内存之间直接传送，仅在传送开始和结束时需要 CPU 介入（预处理和后处理），传送过程中由 DMA 控制器接管总线、直接完成批量数据传输，避免了频繁中断 CPU 所带来的开销，能够满足高速设备的数据传输需求。

四、设计题（2×10=20分）

1.

解答：主存储器扩展与片选逻辑设计（文字推导）

> 以下解答逐字对照 gen.py exec 数据流输出转写，不重算。

基本参数与容量计算

参数	值
CPU 地址总线	16 位（A15~A0）
CPU 数据总线	16 位（D15~D0）
主存总容量	65536 字节 = 64KB
ROM 分区	0000H~3FFFH，16384 字节 = 16KB
RAM 分区	4000H~FFFFH，49152 字节 = 48KB
ROM 芯片规格	8K × 8 位（单片 8192 字节）
RAM 芯片规格	16K × 8 位（单片 16384 字节）

容量守恒验证：ROM 分区 16384 字节 + RAM 分区 49152 字节 = 65536 字节 = 主存总容量 √

第 1 问（3 分）：ROM 区和 RAM 区所需芯片数量

ROM 区

- **位扩展** = CPU 数据总线宽度 ÷ ROM 芯片位宽 = 16 ÷ 8 = **2 倍**
- 原因：ROM 芯片为 8 位，数据总线为 16 位，需 2 片并联才能提供 16 位数据宽度。
- **字扩展** = ROM 分区容量 ÷ ROM 芯片字数 = 16384 ÷ 8192 = **2 倍**
- 原因：ROM 分区 16KB，单个 8K×8 位芯片提供 8K 个地址单元，需 2 组字扩展。
- **ROM 区总芯片数** = 字扩展 2 × 位扩展 2 = **4 片**
- 划分为 **2 个芯片组**，每组 2 片（位扩展并联）。

RAM 区

- **位扩展** = CPU 数据总线宽度 ÷ RAM 芯片位宽 = 16 ÷ 8 = **2 倍**
- 原因：RAM 芯片为 8 位，数据总线为 16 位，需 2 片并联。
- **字扩展** = RAM 分区容量 ÷ RAM 芯片字数 = 49152 ÷ 16384 = **3 倍**
- 原因：RAM 分区 48KB，单个 16K×8 位芯片提供 16K 个地址单元，需 3 组字扩展。
- **RAM 区总芯片数** = 字扩展 3 × 位扩展 2 = **6 片**
- 划分为 **3 个芯片组**，每组 2 片（位扩展并联）。

关键点：位扩展芯片在同一芯片组内并联，共享同一条片选信号，不额外占用地址线（场景规则 5）。

第 2 问（2 分）：片内地址线与高位地址线

ROM 芯片（8K × 8 位）

- **片内地址线** = $\log_2(8192)$ = **13 根**，即 A12~A0
- 8K = 2^{13}，需 13 根地址线寻址片内 8192 个单元。
- **高位地址线（片选译码用）** = 16 − 13 = **3 根**，即 A15、A14、A13

RAM 芯片（16K × 8 位）

- **片内地址线** = $\log_2(16384)$ = **14 根**，即 A13~A0
- 16K = 2^{14}，需 14 根地址线寻址片内 16384 个单元。
- **高位地址线（片选译码用）** = 16 − 14 = **2 根**，即 A15、A14

ROM 与 RAM 地址线分组的差异及原因

芯片类型	片内地址线	高位地址线
ROM（8K×8位）	13 根（A12~A0）	3 根（A15~A13）

芯片类型	片内地址线	高位地址线
RAM（16K×8位）	14 根（A13~A0）	2 根（A15~A14）

差异原因：ROM 芯片容量 8K（2^{13}）比 RAM 芯片容量 16K（2^{14}）少一位地址线。因此 ROM 芯片的片内地址线少一根（13 根 vs 14 根），高位地址线多一根（3 根 vs 2 根）参与片选译码。位扩展不额外占用地址线（场景规则5），所以分组仅由芯片字数决定。

第 3 问（3 分）：各芯片组片选信号逻辑表达式

片选信号低电平有效，记为 \overline{CS}。每个芯片组的片选由高位地址线译码产生（片内地址线不参与片选，场景规则6）。

ROM 区（高位地址 A15、A14、A13 参与译码）

芯片组	片选信号逻辑表达式	地址范围
ROM 组0	$\overline{CS}_{ROM0} = \overline{A15} \cdot \overline{A14} \cdot \overline{A13}$	0000H ~ 1FFFH
ROM 组1	$\overline{CS}_{ROM1} = \overline{A15} \cdot \overline{A14} \cdot A13$	2000H ~ 3FFFH

说明：ROM 组0 对应 A15A14A13 = 000（地址 0000H~1FFFH），ROM 组1 对应 A15A14A13 = 001（地址 2000H~3FFFH）。每个芯片组内 2 片 ROM 通过位扩展并联，共享同一条片选信号。

RAM 区（高位地址 A15、A14 参与译码）

芯片组	片选信号逻辑表达式	地址范围
RAM 组0	$\overline{CS}_{RAM0} = \overline{A15} \cdot A14$	4000H ~ 7FFFH
RAM 组1	$\overline{CS}_{RAM1} = A15 \cdot \overline{A14}$	8000H ~ BFFFH
RAM 组2	$\overline{CS}_{RAM2} = A15 \cdot A14$	C000H ~ FFFFH

说明：RAM 组0 对应 A15A14 = 01（地址 4000H~7FFFH），RAM 组1 对应 A15A14 = 10（地址 8000H~BFFFH），RAM 组2 对应 A15A14 = 11（地址 C000H~FFFFH）。每个芯片组内 2 片 RAM 通过位扩展并联，共享同一条片选信号。

第 4 问（2 分）：各芯片组地址范围与验证

全部芯片组地址范围（按地址从小到大排列）

序号	芯片组	起始地址	结束地址	容量
0	ROM 组0	0000H	1FFFH	8KB

序号	芯片组	起始地址	结束地址	容量
1	ROM 组1	2000H	3FFFH	8KB
2	RAM 组0	4000H	7FFFH	16KB
3	RAM 组1	8000H	BFFFH	16KB
4	RAM 组2	C000H	FFFFH	16KB

连续性验证

- 组0 末址 1FFFH + 1 = 2000H = 组1 首址 √
- 组1 末址 3FFFH + 1 = 4000H = 组2 首址 √
- 组2 末址 7FFFH + 1 = 8000H = 组3 首址 √
- 组3 末址 BFFFH + 1 = C000H = 组4 首址 √

全覆盖验证

- 首地址 = 0000H（主存空间起始）√
- 末地址 = FFFFH（主存空间结束）√
- 总覆盖范围 = 0000H ~ FFFFH = 主存全部地址空间 √
- 各芯片组地址范围两两交集为空，无重叠 √

最终结论

1. **芯片数量**：ROM 区 4 片（2 字扩展 × 2 位扩展），RAM 区 6 片（3 字扩展 × 2 位扩展），共 10 片。
2. **地址线分组**：ROM 片内 13 根（A12~A0）/ 高位 3 根（A15~A13）；RAM 片内 14 根（A13~A0）/ 高位 2 根（A15~A14）。
3. **片选逻辑**：ROM 用 A15A14A13 译码产生 2 个片选，RAM 用 A15A14 译码产生 3 个片选，位扩展芯片共享同一片选。
4. **地址范围**：5 个芯片组从 0000H 到 FFFFH 连续覆盖，无重叠、无遗漏。

2.

数据通路与指令执行微操作序列 — 解答

指令语义

`SUB R1, R2`：执行 **R1 ← R1 − R2**。

ALU 结构：输入 A 来自锁存器 Y，输入 B 来自总线 Bus，运算结果送入锁存器 Z，支持 ADD（A+B）和 SUB（A−B）。因此被减数 R1 必须先存入 Y，减数 R2 经总线送入 ALU 输入 B。

子问题1（4分）：取指阶段微操作序列

节拍	功能描述	有效控制信号
T1	PC 内容送入 MAR	PCout, MARin
T2	存储器读：M[MAR] 的内容读入 MDR	Read
T3	MDR 内容送入 IR	MDRout, IRin
T4	PC 自增（内部逻辑，不占用总线）	PC+1

取指阶段分析：

- **T1**：PC 内容经总线送入 MAR，为存储器访问提供地址。PCout 驱动总线，MARin 接收写入 MAR。
- **T2**：Read 信号有效，存储器将 M[MAR] 的内容读入 MDR。此节拍无总线驱动操作（存储器自动写入 MDR）。
- **T3**：MDR 内容经总线送入 IR，完成取指令。MDRout 驱动总线，IRin 接收写入 IR。
- **T4**：PC+1 信号使 PC 内容自增 1，指向下一条指令地址。该操作由 PC 内部自增逻辑完成，不占用总线。

取指阶段共 4 个节拍（T1~T4），完成了 PC→MAR、存储器读、MDR→IR、PC 自增四项操作。

子问题2（6分）：执行阶段微操作序列

节拍	功能描述	有效控制信号
T5	R1 内容送入 ALU 输入锁存器 Y（被减数）	R1out, Yin
T6	R2 内容送总线（ALU 输入B），执行减法，结果存入 Z	R2out, SUB, Zin
T7	Z 内容经总线送入 R1（写回结果）	Zout, R1in

执行阶段分析：

- **T5**：R1 内容（被减数）经总线送入 ALU 输入锁存器 Y。R1out 驱动总线，Yin 接收写入 Y。此时 ALU 输入 A = R1。
- **T6**：R2 内容（减数）经总线送入 ALU 输入 B，SUB 信号使 ALU 执行 A−B = R1−R2，运算结果由 Zin 写入锁存器 Z。R2out 驱动总线（ALU 输入 B 直接来自总线），Zin 接收运算结果写入 Z。

· **T7**：Z 内容（运算结果 R1−R2）经总线送回 R1，完成写回。Zout 驱动总线，R1in 接收写入 R1。

执行阶段共 3 个节拍（T5~T7），完成了取操作数（R1→Y、R2→Bus）、ALU 减法运算（R1−R2→Z）、结果写回（Z→R1）。

通过条件验证

全部指令共 7 个节拍（T1~T7），连续编号，取指阶段 T1~T4 与执行阶段 T5~T7 衔接。

通过条件	验证结果
条件1 总线无冲突	每个节拍至多 1 个 "out" 信号：T1=PCout, T2=无, T3=MDRout, T4=无, T5=R1out, T6=R2out, T7=Zout √
条件2 控制信号一致性	使用的全部 13 个信号均来自 C1~C15 定义表，无未定义信号 √
条件3 取指完整性	PC→MAR(T1) + Read(T2) + MDR→IR(T3) + PC+1(T4) 四项齐全 √
条件4 减法操作数顺序	R1→Y(T5, 被减数送ALU输入A), R2→Bus(T6, 减数送ALU输入B), 运算 Y−Bus = R1−R2 √
条件5 写回正确性	结果存入 Z(T6, Zin) → Z→Bus→R1(T7, Zout+R1in), T7 > T6, 未跳过 Z 锁存器 √
条件6 节拍连续性	T1~T7 连续编号，跨阶段衔接 T4→T5 无遗漏 √

使用的全部控制信号（13个）：PCout, MARin, Read, MDRout, IRin, PC+1, R1out, Yin, R2out, SUB, Zin, Zout, R1in。